

به نام خدا



دانشگاه بو علی سینا

unmatched project

اعضا گروه:

زهرا اکبری

سونیا ساطع

استاد درس : دکتر یوسف صنعتی

فهرست

1. مقدمه
- Card.2
- Fighter.3
- FighterDefinition.4
- Player.5
- Factories.6
- Board.7
- Space.8
- GameController.9
- TUIApp .10
- GameException .11
12. چالش‌های پروژه
13. منابع

مقدمه

این پروژه با هدف طراحی و پیاده‌سازی نسخه‌ای دیجیتال از بازی رومیزی **Unmatched** توسعه یافته است. در این بازی، دو بازیکن با انتخاب قهرمانان منحصربه‌فرد و استفاده از کارت‌ها، توانایی‌های ویژه و جای‌گیری مناسب روی نقشه، در یک نبرد استراتژیک با یکدیگر رقابت می‌کنند. هدف اصلی بازی، شکست دادن قهرمان حریف از طریق مدیریت صحیح حرکت، حمله، دفاع و استفاده به‌موقع از قابلیت‌های هر شخصیت است.

در این پروژه، قوانین اصلی بازی شامل ساختار نوبت، سیستم حرکت، حمله و دفاع، مدیریت کارت‌ها، نواحی نقشه، توانایی‌های ویژه شخصیت‌ها و سایر مکانیک‌های بازی به‌صورت دیجیتال پیاده‌سازی شده‌اند تا تجربه‌ای نزدیک به نسخه رومیزی بازی فراهم شود.

Card

انواع داده شمارشی (Enumerations)

در ابتدای فایل چند نوع داده شمارشی تعریف شده است که نقش مهمی در مدل‌سازی اطلاعات بازی دارند.

HeroKind

این شمارش نوع قهرمان‌های اصلی بازی را مشخص می‌کند.

مقادیر موجود:

- Dracula
- Sherlock

کاربرد این نوع داده، تشخیص شخصیت اصلی هر بازیکن در بخش‌های مختلف پروژه است

Character

این شمارش تمامی شخصیت‌هایی که می‌توانند مالک یک کارت باشند را مشخص می‌کند.

مقادیر:

- Any
- Dracula
- Sister
- Sherlock
- Watson

وجود مقدار **Any** نشان می‌دهد که برخی کارت‌ها محدود به شخصیت خاصی نیستند.

CardType

نوع عملکرد هر کارت را مشخص می‌کند.

مقادیر:

- Attack •
- Defense •
- Versatile •
- Scheme •

این مقدار در بخش‌های مختلف پروژه برای تعیین قابلیت حمله یا دفاع کارت استفاده می‌شود

Timing

زمان فعال شدن اثر کارت را مشخص می‌کند.

مقادیر:

- None •
- Immediately •
- DuringCombat •
- AfterCombat •

این اطلاعات هنگام اجرای افکت‌های کارت مورد استفاده قرار می‌گیرد.

EffectId

این شمارش شناسه تمامی افکت‌های موجود در بازی را نگهداری می‌کند.

نمونه‌هایی از مقادیر:

- None •
- DraculaBloodStrike •
- DraculaMistForm •
- SherlockCounterpunch •
- WatsonAid •
- SherlockElementary •
- SherlockGameAfoot •
- ... •

به جای ذخیره مستقیم منطق هر افکت، تنها شناسه آن نگهداری می‌شود تا در سایر بخش‌های پروژه بر اساس این شناسه، رفتار مناسب اجرا گردد

کلاس Card

هدف و مسئولیت کلاس

کلاس Card نمایش‌دهنده یک کارت در بازی است. این کلاس تمامی اطلاعات مرتبط با یک کارت شامل مشخصات ظاهری، قدرت‌ها، نوع کارت، افکت و توضیحات آن را نگهداری کرده و متدهایی برای دسترسی و بررسی ویژگی‌های کارت ارائه می‌دهد.

ویژگی‌ها (Attributes)

_title

نام کارت را ذخیره می‌کند.

_owner

شخصیتی که مالک این کارت است را مشخص می‌کند.

_type

نوع کارت (حمله، دفاع، چندمنظوره یا طرح) را نگهداری می‌کند.

_attack

قدرت حمله کارت را ذخیره می‌کند.

_defense

قدرت دفاع کارت را نگهداری می‌کند.

_boost

مقدار افزایش قدرت (Boost Value) کارت را مشخص می‌کند.

_timing

زمان فعال شدن اثر کارت را ذخیره می‌کند.

_effect

شناسه افکت مربوط به کارت را نگهداری می‌کند.

_effectText

توضیح متنی افکت کارت را ذخیره می‌کند.

متدهای کلاس

سازنده

Card

وظیفه کلی

مقداردهی اولیه تمامی ویژگی‌های کارت هنگام ایجاد شیء.

عملکرد

تمام اطلاعات دریافت‌شده را در ویژگی‌های داخلی کلاس ذخیره می‌کند. برای رشته‌ها از **std::move** استفاده شده تا عملیات انتقال داده انجام شود و از کپی غیرضروری جلوگیری گردد.

محل استفاده

در زمان ایجاد کارت‌های بازی.

منطق پیاده‌سازی

تمام پارامترهای ورودی مستقیماً در اعضای کلاس مقداردهی می‌شوند.

()title

وظیفه

بازگرداندن نام کارت.

عملکرد

مرجع ثابت رشته مربوط به عنوان کارت را برمی‌گرداند.

()owner

وظیفه

دریافت شخصیت مالک کارت.

عملکرد

مقدار عضو **owner_** را بازمی‌گرداند.

()type

وظیفه

بازگرداندن نوع کارت.

()attack

وظیفه

بازگرداندن قدرت حمله کارت.

()defense

وظیفه

بازگرداندن قدرت دفاع کارت.

()boost

وظیفه

بازگرداندن مقدار Boost کارت.

()timing

وظیفه

بازگرداندن زمان اجرای افکت.

()effect

وظیفه

بازگرداندن شناسه افکت کارت.

()effectText

وظیفه

بازگرداندن توضیح متنی افکت کارت.

()canAttack

وظیفه کلی

بررسی قابلیت حمله کارت.

عملکرد

اگر نوع کارت **Attack** یا **Versatile** باشد مقدار **true** بازگردانده می‌شود.

محل استفاده

در بخش‌هایی که امکان اجرای حمله بررسی می‌شود.

منطق پیاده‌سازی

نوع کارت با دو مقدار مشخص مقایسه شده و نتیجه منطقی بازگردانده می‌شود.

()canDefend

وظیفه کلی

بررسی قابلیت دفاع کارت.

عملکرد

در صورتی که نوع کارت **Defense** یا **Versatile** باشد مقدار **true** برگردانده می‌شود.

()isScheme

وظیفه کلی

تشخیص اینکه کارت از نوع **Scheme** است یا خیر.

عملکرد

نوع کارت را با **CardType::Scheme** مقایسه می‌کند.

()hasCombatEffect

وظیفه کلی

بررسی وجود افکت رزمی.

عملکرد

اگر کارت دارای افکت معتبر بوده و از نوع Scheme نباشد، مقدار **true** برگردانده می‌شود.

منطق پیاده‌سازی

دو شرط هم‌زمان بررسی می‌شوند:

- افکت نباید **None** باشد.
- نوع کارت نباید **Scheme** باشد.

()typeLabel

وظیفه کلی

تبدیل نوع کارت به متن قابل نمایش.

عملکرد

با استفاده از دستور **switch**، متن مناسب برای هر نوع کارت را تولید می‌کند.

()ownerLabel

وظیفه کلی

نمایش نام شخصیت مالک کارت.

عملکرد

با استفاده از تابع **characterName**، مقدار شمارشی شخصیت را به رشته متنی تبدیل می‌کند.

()timingLabel

وظیفه کلی

تبدیل زمان اجرای افکت به متن.

عملکرد

برای هر مقدار موجود در **Timing** متن مناسب را بازمی‌گرداند

توابع سراسری

(heroKindName(HeroKind hero

وظیفه کلی

تبدیل نوع قهرمان به رشته قابل نمایش.

عملکرد

بر اساس مقدار **HeroKind** نام مناسب را برمی‌گرداند.

محل استفاده

در بخش‌هایی که لازم است نام قهرمان برای کاربر نمایش داده شود.

منطق پیاده‌سازی

با استفاده از **switch** برای هر مقدار، رشته متناظر بازگردانده می‌شود.

(characterName(Character character

وظیفه کلی

تبدیل نوع شخصیت به متن.

عملکرد

برای هر مقدار موجود در **Character**، نام متنی مناسب را تولید می‌کند.

محل استفاده

در نمایش اطلاعات شخصیت‌ها و همچنین متد **ownerLabel()**.

منطق پیاده‌سازی

از ساختار **switch** برای نگاشت مقادیر شمارشی به رشته‌های متنی استفاده شده است.

ارتباط کلاس با سایر کلاس‌ها

- کلاس **Card** از انواع داده شمارشی تعریف‌شده در همین فایل برای نگهداری اطلاعات کارت استفاده می‌کند.
- تابع **ownerLabel()** به تابع سراسری **characterName()** وابسته است.
- شناسه‌های موجود در **EffectId** زمینه ارتباط این کلاس با بخش مدیریت افکت‌های بازی را فراهم می‌کنند، اما منطق اجرای افکت‌ها در این فایل پیاده‌سازی نشده است.
- این کلاس به‌عنوان مدل داده‌ای کارت، توسط سایر اجزای پروژه برای ایجاد، نگهداری و پردازش کارت‌های بازی مورد استفاده قرار می‌گیرد.

Fighter

هدف فایل

این فایل کلاس **Fighter** را پیاده‌سازی می‌کند. این کلاس نماینده یک مبارز (**Fighter**) در بازی است و وضعیت جاری هر شخصیت را در طول بازی مدیریت می‌کند. اطلاعاتی مانند میزان سلامتی، موقعیت روی نقشه و عملیات مربوط به دریافت آسیب، درمان، قرارگیری روی صفحه و حذف از صفحه توسط این کلاس مدیریت می‌شوند.

کلاس **Fighter** از اطلاعات ثابت موجود در **FighterDefinition** استفاده کرده و وضعیت متغیر مبارز را در طول اجرای بازی نگهداری می‌کند.

هدف و مسئولیت کلاس

کلاس **Fighter** مسئول نگهداری وضعیت یک مبارز در جریان بازی است. این کلاس علاوه بر دسترسی به مشخصات ثابت مبارز، وضعیت جاری او را نیز مدیریت می‌کند. مهمترین مسئولیت‌های این کلاس عبارت‌اند از:

- نگهداری وضعیت سلامتی مبارز
- نگهداری موقعیت فعلی روی نقشه
- اعمال آسیب
- درمان مبارز
- احیای مبارز
- حذف مبارز از صفحه بازی
- ارائه اطلاعات مربوط به مبارز به سایر بخش‌های پروژه

ویژگی‌های کلاس (Attributes)

ویژگی	توضیح
_definition	مشخصات ثابت مبارز شامل شناسه، نام، میزان حرکت، محدوده حمله، سلامتی اولیه و سایر ویژگی‌های تعریف‌شده در FighterDefinition

میزان سلامتی فعلی مبارز	_health
شناسه فضای فعلی مبارز روی نقشه. مقدار - 1 نشان‌دهنده عدم حضور روی صفحه است.	_spaceId

متدهای کلاس

(Fighter(FighterDefinition definition

وظیفه

سازنده کلاس.

عملکرد

مشخصات مبارز را دریافت کرده و سلامتی اولیه را برابر مقدار تعریف‌شده در **FighterDefinition** قرار می‌دهد. همچنین در ابتدای ایجاد شیء، مبارز هنوز روی صفحه قرار ندارد و مقدار **_spaceId** برابر - 1 تنظیم می‌شود.

محل استفاده در پروژه

در زمان ایجاد مبارزان هر بازیکن.

منطق پیاده‌سازی

شیء **FighterDefinition** با استفاده از **std::move** منتقل شده و سپس مقدار سلامت و موقعیت اولیه مقداردهی می‌شود.

()definition

وظیفه

دسترسی به مشخصات ثابت مبارز.

عملکرد

مرجع مشخصات ذخیره‌شده در `_definition` را بازی‌گرداند.

محل استفاده

هر بخشی که به اطلاعات اصلی مبارز نیاز داشته باشد.

`()id`

وظیفه

دریافت شناسه مبارز.

عملکرد

شناسه ذخیره‌شده در `FighterDefinition` را بازی‌گرداند.

`()displayName`

وظیفه

دریافت نام نمایشی مبارز.

عملکرد

نام قابل نمایش مبارز را بازی‌گرداند.

`()cardOwner`

وظیفه

دریافت مالک کارت‌های مبارز.

عملکرد

مقدار ویژگی `Character` مربوط به مبارز را برمی‌گرداند.

()isHero

وظیفه

تشخیص قهرمان بودن مبارز.

عملکرد

مشخص می‌کند مبارز از نوع Hero است یا خیر.

()maxHealth

وظیفه

دریافت حداکثر سلامتی.

عملکرد

حداکثر مقدار سلامتی تعریف‌شده برای مبارز را بازمی‌گرداند.

()health

وظیفه

دریافت سلامتی فعلی.

عملکرد

مقدار فعلی **health** را بازمی‌گرداند.

()move

وظیفه

دریافت میزان حرکت مبارز.

عملکرد

مقدار حرکت تعریف شده در مشخصات مبارز را بازمی گرداند.

()range

وظیفه

دریافت نوع حمله مبارز.

عملکرد

نوع برد حمله (**AttackRange**) را بازمی گرداند.

()spaceId

وظیفه

دریافت موقعیت فعلی مبارز.

عملکرد

شناسه فضای فعلی مبارز را برمی گرداند.

()defeated

وظیفه

بررسی شکست مبارز.

عملکرد

در صورتی که میزان سلامتی کمتر یا مساوی صفر باشد، مقدار **true** بازمی گرداند.

(placeAt(int spaceId

وظیفه

قرار دادن مبارز روی صفحه.

عملکرد

موقعیت جدید مبارز را ثبت می‌کند.

محل استفاده

در زمان شروع بازی یا جابجایی مبارز.

منطق پیاده‌سازی

در ابتدا بررسی می‌شود مبارز شکست‌خورده نباشد. در صورت شکست، استثناء **RuleViolation** ایجاد می‌شود؛ در غیر این صورت شناسه فضای جدید ذخیره می‌شود.

(damage(int amount

وظیفه

اعمال آسیب.

عملکرد

از میزان سلامتی مبارز کم می‌کند.

محل استفاده

در هنگام اجرای حمله‌ها.

منطق پیاده‌سازی

ابتدا مقدار آسیب بررسی می‌شود تا منفی نباشد. سپس سلامتی کاهش یافته و مقدار آن کمتر از صفر نمی‌شود. اگر سلامتی به صفر برسد، مبارز از صفحه حذف شده و مقدار **spaceId** برابر **1** قرار می‌گیرد.

(heal(int amount

وظیفه

درمان مبارز.

عملکرد

سلامتی مبارز را افزایش می‌دهد.

محل استفاده

در اثر کارت‌ها یا قابلیت‌های درمانی.

منطق پیاده‌سازی

در ابتدا مقدار درمان بررسی می‌شود. اگر مقدار منفی باشد، استثناء ایجاد می‌شود. در صورتی که مبارز شکست خورده باشد، عملی انجام نمی‌شود. در غیر این صورت سلامتی افزایش یافته و از مقدار حداکثر بیشتر نمی‌شود.

(reviveAt(int spaceId

وظیفه

احیای مبارز.

عملکرد

مبارز را با حداقل یک واحد سلامتی به صفحه بازی می‌گرداند.

محل استفاده

در قابلیت‌هایی که امکان بازگرداندن مبارز به بازی را فراهم می‌کنند.

منطق پیاده‌سازی

سلامتی حداقل برابر یک قرار داده شده و موقعیت جدید مبارز ثبت می‌شود.

() removeFromBoard

وظیفه

حذف مبارز از صفحه.

عملکرد

مقدار `spaceId` را برابر -1 قرار می‌دهد تا نشان دهد مبارز دیگر روی نقشه حضور ندارد.

محل استفاده

در شرایطی که مبارز باید بدون تغییر سلامتی از صفحه حذف شود.

منطق پیاده‌سازی

تنها شناسه موقعیت مبارز را تغییر می‌دهد.

ارتباط کلاس با سایر کلاس‌ها

کلاس **Fighter** با کلاس‌ها و انواع زیر در ارتباط است:

- **FighterDefinition** (منبع اطلاعات ثابت مبارز)
- **GameException**
- **RuleViolation**
- **Character**
- **AttackRange**

این کلاس توسط سایر بخش‌های منطق بازی مانند بازیکن، کنترل‌کننده بازی و سیستم مبارزه برای مدیریت وضعیت مبارزان مورد استفاده قرار می‌گیرد.

وابستگی‌های فایل

کتابخانه‌های استاندارد

- **algorithm** : برای استفاده از توابع **std::min** و **std::max**
- **utility** : برای استفاده از **std::move**

فایل‌های داخلی پروژه

- **FighterDefinition.hpp**
- **GameExceptions.hpp**

FighterDefinition

هدف فایل

این فایل کلاس **FighterDefinition** را تعریف و پیاده‌سازی می‌کند. وظیفه این کلاس نگهداری مشخصات ثابت و اولیه هر مبارز (Fighter) است؛ مشخصاتی که در طول اجرای بازی تغییر نمی‌کنند. این اطلاعات هنگام ایجاد شیء **Fighter** مورد استفاده قرار گرفته و به عنوان مرجع ویژگی‌های اصلی هر مبارز عمل می‌کنند.

کلاس

FighterDefinition

هدف و مسئولیت کلاس

کلاس **FighterDefinition** مسئول ذخیره اطلاعات پایه و تغییرناپذیر مربوط به هر مبارز است. این اطلاعات شامل شناسه، نام نمایشی، مالک کارت‌ها، نوع مبارز، میزان سلامتی اولیه، مقدار حرکت، نوع حمله و سبک مبارز است.

این کلاس تنها نقش نگهداری اطلاعات را دارد و هیچ تغییری در وضعیت مبارز طی بازی ایجاد نمی‌کند.

ویژگی‌های کلاس (Attributes)

ویژگی	توضیح
_id	شناسه یکتای مبارز
_displayName	نام نمایشی مبارز
_cardOwner	مشخص می‌کند کارت‌های مبارز متعلق به کدام شخصیت هستند

مشخص می‌کند مبارز از نوع Hero است یا خیر	_hero
حداکثر میزان سلامتی مبارز	_maxHealth
تعداد واحدهای حرکت مبارز	_move
نوع برد حمله (Melee یا Ranged)	_range
سبک یا ویژگی متنی مبارز	_style

متدهای کلاس

FighterDefinition(...)

وظیفه

سازنده کلاس.

عملکرد

تمام مشخصات اولیه مبارز را دریافت کرده و در ویژگی‌های کلاس ذخیره می‌کند.

محل استفاده در پروژه

در زمان تعریف یا ایجاد مشخصات اولیه مبارزان.

منطق پیاده‌سازی

پارامترهای رشته‌ای با استفاده از `std::move` منتقل شده و سایر مقادیر مستقیماً در ویژگی‌های متناظر ذخیره می‌شوند.

id()

وظیفه

دریافت شناسه مبارز.

عملکرد

شناسه ذخیره شده در **id_** را بازمی‌گرداند.

محل استفاده

در بخش‌هایی که نیاز به شناسایی یکتای مبارز وجود دارد.

displayName()

وظیفه

دریافت نام نمایشی مبارز.

عملکرد

نام قابل نمایش مبارز را بازمی‌گرداند.

cardOwner()

وظیفه

دریافت مالک کارت‌های مبارز.

عملکرد

مقدار ویژگی **Character** مربوط به مالک کارت‌ها را بازمی‌گرداند.

isHero()

وظیفه

بررسی Hero بودن مبارز.

عملکرد

مقدار ویژگی `hero_` را بازمی‌گرداند.

`maxHealth()`

وظیفه

دریافت حداکثر سلامتی.

عملکرد

مقدار `maxHealth_` را بازمی‌گرداند.

`move()`

وظیفه

دریافت میزان حرکت.

عملکرد

تعداد واحدهای حرکت تعریف‌شده برای مبارز را بازمی‌گرداند.

`range()`

وظیفه

دریافت نوع حمله.

عملکرد

مقدار `AttackRange` مربوط به مبارز را بازمی‌گرداند.

()style

وظیفه

دریافت سبک مبارز.

عملکرد

رشته ذخیره شده در `_style` را بازمی‌گرداند.

()rangeLabel

وظیفه

تبدیل نوع حمله به متن قابل نمایش.

عملکرد

در صورتی که نوع حمله `AttackRange::Melee` باشد، رشته `"Melee"` و در غیر این صورت `"Ranged"` را بازمی‌گرداند.

محل استفاده

در بخش‌هایی از پروژه که نیاز به نمایش نوع حمله برای کاربر وجود داشته باشد.

منطق پیاده‌سازی

با استفاده از یک شرط ساده، مقدار شمارشی `AttackRange` به متن قابل نمایش تبدیل می‌شود.

نوع داده

AttackRange

هدف

نوع شمارشی `AttackRange` نوع برد حمله هر مبارز را مشخص می‌کند.

مقادیر تعریف‌شده

مقدار	توضیح
Melee	حمله نزدیک
Ranged	حمله از فاصله دور

این نوع داده در مشخصات مبارز ذخیره شده و توسط سایر بخش‌های پروژه برای تعیین نحوه حمله مورد استفاده قرار می‌گیرد.

ارتباط کلاس با سایر کلاس‌ها

کلاس `FighterDefinition` با اجزای زیر در ارتباط است:

- `Fighter` (برای ایجاد و مقداردهی مشخصات اولیه مبارز)
- `Character` (جهت تعیین مالک کارت‌های مبارز)
- `AttackRange` (برای تعیین نوع برد حمله)

این کلاس اطلاعات پایه‌ای را در اختیار کلاس `Fighter` قرار می‌دهد و توسط بخش‌های مختلف منطق بازی برای دسترسی به مشخصات ثابت مبارزان استفاده می‌شود.

وابستگی‌های فایل

کتابخانه‌های استاندارد

- `string` : برای نگهداری رشته‌ها
- `utility` : برای استفاده از `std::move`

فایل‌های داخلی پروژه

- `Card.hpp` : برای استفاده از نوع `Character`

Player

هدف فایل

این فایل مسئول تعریف و پیاده‌سازی کلاس **Player** است. این کلاس اطلاعات مربوط به هر بازیکن را نگهداری کرده و مدیریت اجزای وابسته به او، شامل مبارزان، دسته کارت‌ها، کارت‌های دست و کارت‌های دورریخته‌شده را بر عهده دارد. همچنین متدهایی برای جستجوی مبارزان، مدیریت کارت‌ها و بررسی وضعیت شخصیت‌های زنده در اختیار سایر بخش‌های پروژه قرار می‌دهد.

کلاس Player

هدف و مسئولیت کلاس

کلاس **Player** نماینده یک بازیکن در بازی است. این کلاس علاوه بر نگهداری اطلاعات پایه بازیکن، مجموعه مبارزان و کارت‌های متعلق به او را مدیریت کرده و عملیات مرتبط با دسترسی به مبارزان، جابه‌جایی کارت‌ها و بررسی وضعیت شخصیت‌ها را انجام می‌دهد.

ویژگی‌ها (Attributes)

_id

شناسه بازیکن را ذخیره می‌کند.

_name

نام بازیکن را نگهداری می‌کند.

_age

سن بازیکن را ذخیره می‌کند.

_hero

نوع قهرمان انتخاب شده توسط بازیکن را مشخص می‌کند.

_fighters

مجموعه مبارزان متعلق به بازیکن را نگهداری می‌کند.

_deck

دسته کارت‌های بازیکن را ذخیره می‌کند.

_hand

کارت‌های موجود در دست بازیکن را نگهداری می‌کند.

_discardPile

کارت‌های استفاده شده یا دور ریخته شده بازیکن را ذخیره می‌کند.

متدهای کلاس

سازنده

Player

وظیفه کلی

ایجاد یک بازیکن و مقداردهی اطلاعات اولیه.

عملکرد

شناسه، نام، سن و قهرمان بازیکن را مقداردهی می‌کند.

محل استفاده

در زمان ایجاد بازیکنان بازی.

منطق پیاده‌سازی

اطلاعات دریافت‌شده مستقیماً در اعضای کلاس ذخیره می‌شوند و برای رشته نام از `std::move` استفاده شده است.

`()id`

وظیفه

بازگرداندن شناسه بازیکن.

`()name`

وظیفه

بازگرداندن نام بازیکن.

`()age`

وظیفه

بازگرداندن سن بازیکن.

`()hero`

وظیفه

بازگرداندن نوع قهرمان انتخاب‌شده.

`()fighters`

وظیفه کلی

دسترسی به مجموعه مبارزان بازیکن.

عملکرد

دو نسخه از این متد وجود دارد:

- نسخه قابل تغییر
- نسخه فقط خواندنی (**const**)

محل استفاده

برای مدیریت یا مشاهده مبارزان بازیکن در سایر بخش‌های پروژه.

deck()

وظیفه کلی

دسترسی به دسته کارت‌های بازیکن.

عملکرد

در دو نسخه قابل تغییر و فقط خواندنی پیاده‌سازی شده است.

hand()

وظیفه کلی

دسترسی به کارت‌های موجود در دست بازیکن.

discardPile()

وظیفه کلی

دسترسی به کارت‌های دور ریخته‌شده.

(fighterById(const std::string& fighterId

وظیفه کلی

یافتن مبارز بر اساس شناسه.

عملکرد

مبارزی که شناسه آن با مقدار ورودی برابر باشد را جستجو کرده و بازمی‌گرداند.

محل استفاده

در بخش‌هایی که لازم است یک مبارز مشخص بر اساس شناسه انتخاب شود.

منطق پیاده‌سازی

با استفاده از تابع `std::find_if` تمام مبارزان بررسی می‌شوند. در صورت پیدا نشدن مبارز، استثنای `RuleViolation` پرتاب می‌شود.

این متد در دو نسخه معمولی و `const` پیاده‌سازی شده است.

()heroFighter

وظیفه کلی

دریافت مبارز اصلی (Hero) بازیکن.

عملکرد

اولین مبارزی که ویژگی `isHero()` آن برابر `true` باشد را بازمی‌گرداند.

محل استفاده

در بخش‌هایی که نیاز به دسترسی مستقیم به شخصیت اصلی بازیکن وجود دارد.

منطق پیاده‌سازی

با استفاده از `std::find_if` مبارزان پیمایش می‌شوند. در صورت نبود مبارز اصلی، استثنای `RuleViolation` ایجاد می‌شود.

این متد نیز دارای نسخه معمولی و `const` است.

()aliveFighters

وظیفه کلی

دریافت مبارزان زنده بازیکن.

عملکرد

تمام مبارزانی که شکست نخورده‌اند در یک بردار جدید قرار گرفته و بازگردانده می‌شوند.

محل استفاده

در بررسی وضعیت فعلی بازیکن یا انتخاب مبارزان قابل استفاده.

منطق پیاده‌سازی

تمام اعضای `_fighters` پیمایش شده و تنها مبارزانی که متد `defeated()` آن‌ها مقدار `false` برمی‌گرداند، در نتیجه قرار می‌گیرند.

این متد نیز در دو نسخه معمولی و فقط خواندنی پیاده‌سازی شده است.

(removeCardFromHand(int index

وظیفه کلی

حذف یک کارت از دست بازیکن.

عملکرد

کارت موجود در اندیس مشخص را از دست بازیکن خارج کرده و همان کارت را بازمی‌گرداند.

محل استفاده

هنگام استفاده یا بازی کردن یک کارت.

منطق پیاده‌سازی

ابتدا معتبر بودن اندیس بررسی می‌شود. سپس کارت با استفاده از `at()` دریافت شده، از بردار حذف می‌شود و در پایان بازگردانده می‌شود. اگر اندیس نامعتبر باشد، استثنای `RuleViolation` ایجاد می‌شود.

(addToDiscard(Card card

وظیفه کلی

افزودن کارت به کارت‌های دورریخته‌شده.

عملکرد

کارت دریافت‌شده به انتهای `_discardPile` اضافه می‌شود.

محل استفاده

پس از استفاده یا حذف کارت از بازی.

منطق پیاده‌سازی

برای انتقال شیء از `std::move` استفاده شده است.

(addToHand(Card card

وظیفه کلی

افزودن کارت به دست بازیکن.

عملکرد

کارت جدید را به مجموعه کارت‌های دست اضافه می‌کند.

محل استفاده

در زمان دریافت یا کشیدن کارت از دسته.

منطق پیاده‌سازی

کارت با استفاده از `std::move` به انتهای بردار `_hand` منتقل می‌شود.

(hasLivingCharacter(Character character

وظیفه کلی

بررسی زنده بودن یک شخصیت مشخص.

عملکرد

بررسی می‌کند که آیا شخصیت موردنظر هنوز در میان مبارزان زنده بازنیکن وجود دارد یا خیر.

محل استفاده

در بررسی شرایط اجرای برخی قوانین یا افکت‌های بازی.

منطق پیاده‌سازی

- اگر مقدار ورودی `Character::Any` باشد، تنها بررسی می‌شود که حداقل یک مبارز زنده وجود داشته باشد.
- در غیر این صورت با استفاده از `std::any_of` تمام مبارزان بررسی می‌شوند تا مبارزی که هم شکست نخورده باشد و هم شخصیت آن با مقدار ورودی برابر باشد، پیدا شود.

ارتباط کلاس با سایر کلاس‌ها

- کلاس `Player` از کلاس `Fighter` برای نگهداری مبارزان بازنیکن استفاده می‌کند.
- کلاس `Player` از کلاس `Card` برای مدیریت دسته کارت‌ها، کارت‌های دست و کارت‌های دورریخته‌شده بهره می‌برد.
- مقدار `HeroKind` و `Character` که در فایل `Card.hpp` تعریف شده‌اند، برای تعیین قهرمان بازنیکن و بررسی شخصیت مبارزان استفاده می‌شوند.
- در صورت نقض قوانین بازی، متدهای مختلف این کلاس از استثنای `RuleViolation` تعریف‌شده در فایل `GameExceptions.hpp` استفاده می‌کنند.

نقش کلاس در عملکرد کلی پروژه

کلاس `Player` هسته مدیریت اطلاعات هر بازنیکن در پروژه است. این کلاس تمام داده‌های مربوط به بازنیکن، مبارزان و کارت‌های او را در یک ساختار واحد نگهداری می‌کند و امکانات لازم برای جستجوی مبارزان، مدیریت کارت‌ها و بررسی وضعیت شخصیت‌ها را در اختیار سایر بخش‌های بازی قرار می‌دهد. به همین دلیل، بیشتر عملیات مرتبط با وضعیت هر بازنیکن از طریق این کلاس انجام می‌شود.

Factories

هدف فایل

این فایل شامل سه کلاس کارخانه (**Factory**) است که وظیفه ایجاد بخش‌های اصلی بازی را بر عهده دارند. این کلاس‌ها فرآیند ساخت اشیای پیچیده را متمرکز کرده و ایجاد صفحه بازی، دسته کارت‌ها و مبارزان هر قهرمان را مدیریت می‌کنند.

کلاس‌های موجود در این فایل عبارت‌اند از:

- `BoardFactory`

- DeckFactory
- FighterFactory

کلاس BoardFactory

هدف و مسئولیت کلاس

کلاس **BoardFactory** مسئول ایجاد صفحه بازی است. در نسخه فعلی تنها متدی برای ساخت صفحه **Baskerville Manor** در این کلاس پیاده‌سازی شده است.

ویژگی‌ها (Attributes)

این کلاس هیچ ویژگی (Attribute) ندارد.

متدهای کلاس

createBaskervilleManor()

وظیفه کلی

ایجاد صفحه بازی **Baskerville Manor**.

عملکرد

تمام خانه‌های صفحه بازی را ایجاد کرده و ارتباط میان آن‌ها را مشخص می‌کند، سپس شیء **Board** را باز می‌گرداند.

محل استفاده

در زمان شروع بازی و ایجاد نقشه بازی.

منطق پیاده‌سازی

- ابتدا یک بردار از نوع **Space** ایجاد می‌شود.
- یک تابع محلی (**addSpace**) برای ساده‌تر شدن ایجاد خانه‌ها تعریف شده است.
- برای هر خانه، اطلاعات زیر ثبت می‌شود:
 - شناسه خانه
 - مختصات سطر و ستون

- ناحیه یا نواحی مربوطه (Zones)
- خانه‌های مجاور
- وضعیت مخفی بودن (secret)
- محل شروع (startSlot)
- در پایان، مجموعه خانه‌ها به سازنده کلاس Board منتقل می‌شود.

کلاس DeckFactory

هدف و مسئولیت کلاس

کلاس DeckFactory مسئول ایجاد دسته کارت‌های مربوط به هر قهرمان است. این کلاس با توجه به شخصیت انتخاب‌شده، مجموعه کارت‌های مخصوص همان شخصیت را تولید می‌کند.

ویژگی‌ها (Attributes)

این کلاس هیچ ویژگی داخلی ندارد.

متدهای کلاس

createDeck(HeroKind hero)

وظیفه کلی

ایجاد دسته کارت‌های یک قهرمان.

عملکرد

بر اساس مقدار HeroKind، کارت‌های مناسب شخصیت موردنظر را ایجاد کرده و در قالب یک بردار بازمی‌گرداند.

محل استفاده

در زمان مقداردی اولیه هر بازیکن و آماده‌سازی دسته کارت‌ها.

منطق پیاده‌سازی

ابتدا یک بردار از نوع Card ایجاد می‌شود.

برای جلوگیری از تکرار کد، یک تابع محلی به نام `addCopies` تعریف شده است که وظیفه ایجاد چند نسخه از یک کارت را بر عهده دارد.

در ادامه:

- اگر قهرمان **Dracula** باشد، کارت‌های اختصاصی دراکولا و شخصیت‌های وابسته به او (مانند **Sister**) ایجاد می‌شوند.
- در غیر این صورت، کارت‌های مربوط به **Sherlock Holmes** و **Watson** ساخته می‌شوند.

هر کارت با اطلاعات زیر ایجاد می‌شود:

- عنوان کارت
- مالک کارت
- نوع کارت
- قدرت حمله
- قدرت دفاع
- مقدار **Boost**
- زمان اجرای افکت
- شناسه افکت
- توضیح افکت

در پایان، بردار کامل کارت‌ها بازگردانده می‌شود.

کلاس **FighterFactory**

هدف و مسئولیت کلاس

کلاس **FighterFactory** مسئول ایجاد مبارزان مربوط به هر قهرمان است.

ویژگی‌ها (Attributes)

این کلاس فاقد ویژگی داخلی است.

متدهای کلاس

(createFighters(HeroKind hero

وظیفه کلی

ایجاد مبارزان مربوط به یک قهرمان.

عملکرد

با توجه به قهرمان انتخاب شده، مجموعه مبارزان مناسب را ایجاد کرده و بازی می گردانند.

محل استفاده

در زمان ایجاد بازیکنان و آماده سازی شخصیت های آنها.

منطق پیاده سازی

ابتدا یک بردار از نوع **Fighter** ایجاد می شود.

● اگر قهرمان **Dracula** باشد، چهار مبارز ساخته می شوند:

- Dracula
- Sister I
- Sister II
- Sister III

● در غیر این صورت، دو مبارز ایجاد می شوند:

- Sherlock Holmes
- Dr. Watson

برای هر مبارز ابتدا یک شیء **FighterDefinition** ساخته شده و سپس شیء **Fighter** از روی آن ایجاد می شود.

مشخصات هر مبارز شامل موارد زیر است:

- شناسه
- نام نمایشی
- شخصیت مالک کارت
- Hero بودن یا نبودن
- حداکثر سلامتی
- میزان حرکت
- نوع برد حمله
- سبک مبارزه

در پایان، مجموعه مبارزان بازگردانده می شود.

ارتباط کلاس ها با سایر کلاس ها

BoardFactory

- از کلاس **Board** برای ایجاد صفحه بازی استفاده می کند.
- برای ساخت صفحه، اشیای **Space** را ایجاد کرده و به سازنده **Board** منتقل می کند.

DeckFactory

- از کلاس **Card** برای ایجاد تمامی کارت‌های بازی استفاده می‌کند.
- از انواع داده **HeroKind**، **Character**، **CardType**، **Timing** و **EffectId** که در فایل **Card.hpp** تعریف شده‌اند، بهره می‌برد.

FighterFactory

- از کلاس **FighterDefinition** برای تعریف مشخصات اولیه مبارزان استفاده می‌کند.
- سپس با استفاده از کلاس **Fighter**، اشیای مربوط به مبارزان را ایجاد می‌کند.
- از نوع داده **AttackRange** برای تعیین برد حمله هر مبارز استفاده می‌شود.

نقش کلاس‌ها در عملکرد کلی پروژه

سه کلاس این فایل مسئول مقداردهی اولیه مهم‌ترین اجزای بازی هستند. **BoardFactory** ساختار صفحه بازی را ایجاد می‌کند، **DeckFactory** کارت‌های متناسب با هر قهرمان را تولید می‌کند و **FighterFactory** مبارزان مربوط به هر شخصیت را می‌سازد. به این ترتیب، فرآیند ایجاد داده‌های اولیه بازی در یک محل متمرکز شده و سایر بخش‌های پروژه تنها با فراخوانی این کارخانه‌ها به اطلاعات موردنیاز دسترسی پیدا می‌کنند.

Board

هدف فایل

این فایل کلاس **Board** را تعریف و پیاده‌سازی می‌کند. وظیفه این کلاس مدیریت ساختار نقشه بازی، نگهداری فضاهای (Space) موجود، بررسی ارتباط میان آن‌ها و ارائه عملیات مرتبط با حرکت، همسایگی، موقعیت‌های شروع و نمایش متنی نقشه است.

کلاس **Board** یکی از اجزای اصلی منطق بازی محسوب می‌شود و سایر بخش‌های پروژه برای انجام عملیات مربوط به حرکت و موقعیت مبارزان از آن استفاده می‌کنند.

کلاس

Board

هدف و مسئولیت کلاس

کلاس **Board** مسئول مدیریت نقشه بازی و ارتباط میان فضاهاست. مهمترین مسئولیت‌های این کلاس عبارت‌اند از:

- نگهداری مجموعه فضاها و نقشه
- اعتبارسنجی ساختار نقشه
- بررسی وجود یک فضا
- دسترسی به اطلاعات هر فضا
- بررسی همسایگی برای حرکت و حمله
- تعیین موقعیت‌های شروع بازی
- محاسبه مسیرهای قابل دسترس
- ارائه نمایش متنی نقشه

ویژگی‌های کلاس (Attributes)

ویژگی	توضیح
<code>_spaces</code>	مجموعه تمام فضاها و موجود روی نقشه بازی

متدهای کلاس

`(Board(std::vector<Space> spaces`

وظیفه

سازنده کلاس.

عملکرد

مجموعه فضاها و نقشه را دریافت و در کلاس ذخیره می‌کند.

محل استفاده

در زمان ایجاد برد بازی.

منطق پیاده‌سازی

پس از ایجاد شیء، ساختار نقشه توسط متد `validate()` بررسی می‌شود (طبق طراحی کلاس).

`spaces()`

وظیفه

دریافت تمام فضاهای نقشه.

عملکرد

مرجع مجموعه `_spaces` را بازمی‌گرداند.

`space(int id)`

وظیفه

دریافت اطلاعات یک فضای مشخص.

عملکرد

فضای متناظر با شناسه داده‌شده را بازمی‌گرداند.

محل استفاده

در بخش‌های مختلف پروژه که نیاز به اطلاعات یک فضای مشخص دارند.

`contains(int id)`

وظیفه

بررسی وجود یک فضا در نقشه.

عملکرد

مشخص می‌کند شناسه موردنظر در نقشه وجود دارد یا خیر.

(areAdjacentForCombat(int left, int right

وظیفه

بررسی مجاورت برای حمله.

عملکرد

مشخص می‌کند دو فضای داده‌شده برای انجام حمله مجاور محسوب می‌شوند یا خیر.

(shareZone(int left, int right

وظیفه

بررسی اشتراک ناحیه (Zone).

عملکرد

تعیین می‌کند دو فضا در یک ناحیه مشترک قرار دارند یا خیر.

(startSpaceForSlot(int slot

وظیفه

دریافت فضای شروع.

عملکرد

شناسه فضای مربوط به جایگاه شروع مشخص‌شده را باز می‌گرداند.

()secretPassageSpaces

وظیفه

دریافت فضاهای دارای مسیر مخفی.

عملکرد

تمام فضاهایی که دارای Secret Passage هستند را بازمی‌گرداند.

(movementNeighbors(int id

وظیفه

دریافت همسایه‌های قابل حرکت.

عملکرد

فضاهایی را که از طریق حرکت قابل دسترسی هستند بازمی‌گرداند.

(directNeighbors(int id

وظیفه

دریافت همسایه‌های مستقیم.

عملکرد

همسایه‌های مستقیم یک فضا را برمی‌گرداند.

(spacesSharingAnyZone(int id

وظیفه

یافتن فضاهای هم‌ناحیه.

عملکرد

تمام فضاهایی را که حداقل یک ناحیه مشترک با فضای داده‌شده دارند بازمی‌گرداند.

(...)freeAdjacentSpaces

وظیفه

یافتن فضاهای مجاور آزاد.

عملکرد

همسایه‌هایی را که توسط تابع بررسی اشغال‌شدگی آزاد تشخیص داده شوند، باز می‌گردانند.

محل استفاده

در محاسبات حرکت مبارزان.

reachableSpaces (...)

وظیفه

محاسبه فضاهای قابل دسترس.

عملکرد

تمام فضاهایی را که با تعداد مشخصی حرکت قابل دستیابی هستند محاسبه می‌کند.

محل استفاده

در عملیات حرکت مبارزان.

renderLines (...)

وظیفه

تولید نمایش متنی نقشه.

عملکرد

یک بوم (Canvas) متنی ایجاد کرده و خطوط ارتباطی و نماد هر فضا را روی آن رسم می‌کند.

محل استفاده

در رابط کاربری متنی (TuiApp).

منطق پیاده‌سازی

ابتدا ارتباط میان فضاها رسم می‌شود، سپس برای هر فضا با توجه به وضعیت آن یکی از موارد زیر نمایش داده می‌شود:

- توکن مبارز
- محل شروع (Start Slot)
- مسیر مخفی (Secret Passage)
- شناسه فضا

در پایان مجموعه خطوط متنی نقشه بازگردانده می‌شود.

متدهای خصوصی (Private Methods)

validate()

وظیفه

اعتبارسنجی کامل ساختار نقشه.

عملکرد

پیش از استفاده از نقشه، مجموعه‌ای از قوانین را بررسی می‌کند.

منطق پیاده‌سازی

این متد موارد زیر را کنترل می‌کند:

- خالی نبودن نقشه
- مثبت بودن شناسه فضاها
- یکتا بودن شناسه‌ها
- تکراری نبودن مختصات
- داشتن حداقل یک Zone
- معتبر بودن شماره Start Slot
- وجود دقیقاً یک فضای شروع برای هر بازیکن
- تعریف صحیح مسیرهای مخفی
- عدم همسایگی یک فضا با خودش
- نبود همسایه تکراری
- وجود تمام همسایه‌های معرفی‌شده
- متقارن بودن ارتباط میان فضاها

در صورت نقض هر قانون، استثناء InvalidSetup ایجاد می‌شود.

findSpace(int id)

وظیفه

یافتن یک فضا بر اساس شناسه.

عملکرد

در مجموعه فضاها جستجو کرده و در صورت وجود، اشاره‌گر آن را باز می‌گرداند.

drawEdge(...)

وظیفه

رسم ارتباط میان دو فضا.

عملکرد

خط اتصال بین دو فضای مجاور را روی بوم متنی رسم می‌کند.

منطق پیاده‌سازی

بر اساس اختلاف مختصات، نوع خط (-، |، / یا \) تعیین شده و بین دو فضا رسم می‌شود.

putToken(...)

وظیفه

قرار دادن نماد یک فضا روی نقشه.

عملکرد

رشته نمایشی مربوط به یک فضا یا مبارز را در مختصات مناسب روی بوم قرار می‌دهد.

ارتباط کلاس با سایر کلاس‌ها

کلاس **Board** با اجزای زیر در ارتباط است:

- **Space**
- **InvalidSetup**

• `GameException`

همچنین این کلاس توسط بخش‌هایی مانند `GameController` و `TuiApp` برای مدیریت حرکت مبارزان، بررسی موقعیت‌ها و نمایش نقشه مورد استفاده قرار می‌گیرد.

وابستگی‌های فایل

کتابخانه‌های استاندارد

- `functional` : برای استفاده از توابع شرطی (`std::function`)
- `map` : برای نگهداری موقعیت مبارزان روی نقشه
- `string` : برای تولید خروجی متنی
- `vector` : برای نگهداری مجموعه فضاها

در پیاده‌سازی فایل نیز از کتابخانه‌های استاندارد مانند `algorithm` و `set` برای جستجو، اعتبارسنجی و بررسی یکنایی داده‌ها استفاده شده است.

فایل‌های داخلی پروژه

- `Space.hpp`
- `GameExceptions.hpp` (برای استفاده از `InvalidSetup`)

Space

هدف فایل

این فایل کلاس `Space` را تعریف و پیاده‌سازی می‌کند. هر شیء از این کلاس نماینده یک خانه (`Space`) از صفحه بازی است و اطلاعات ثابت مربوط به آن، از جمله موقعیت، نواحی (`Zones`)، فضاهاى مجاور، مسیر مخفی و محل شروع بازیکنان را نگهداری می‌کند. کلاس `Board` از این کلاس برای ساخت و مدیریت نقشه بازی استفاده می‌کند.

کلاس

Space

هدف و مسئولیت کلاس

کلاس **Space** مسئول نگهداری اطلاعات مربوط به یک خانه از صفحه بازی است. این اطلاعات شامل شناسه فضا، مختصات قرارگیری، نواحی مربوط به آن، فضاهای مجاور، وجود یا عدم وجود مسیر مخفی و محل شروع بازیکنان است. این کلاس تنها داده‌های هر فضا را ذخیره و در اختیار سایر بخش‌های پروژه قرار می‌دهد و مسئول انجام منطق بازی نیست.

ویژگی‌ها (Attributes)

_id

شناسه یکتای فضای بازی.

_row

شماره سطر محل قرارگیری فضا روی نقشه.

_column

شماره ستون محل قرارگیری فضا روی نقشه.

_zones

فهرست ناحیه‌هایی که این فضا به آن‌ها تعلق دارد.

_adjacent

شناسه فضاهای مجاور که امکان حرکت مستقیم به آن‌ها وجود دارد.

_secretPassage

مشخص می‌کند که آیا این فضا دارای مسیر مخفی است یا خیر.

_startSlot

شماره محل شروع بازی برای بازیکنان. در صورت نبود محل شروع، مقدار آن صفر است.

متدهای کلاس

- `()Space`
- `()id`
- `()row`
- `()column`
- `()zones`
- `()adjacent`
- `()hasSecretPassage`
- `()startSlot`
- `()zoneLabel`

ارتباط با سایر کلاس‌ها

- توسط کلاس **Board** برای ساخت و مدیریت صفحه بازی استفاده می‌شود.
- اطلاعات این کلاس در بخش‌های مختلف پروژه، از جمله نمایش نقشه و محاسبه حرکت مبارزان، مورد استفاده قرار می‌گیرد.

نقش کلاس در عملکرد کلی پروژه

کلاس **Space** کوچکترین واحد تشکیل‌دهنده صفحه بازی است. تمام عملیات مربوط به حرکت، بررسی همسایگی، نمایش نقشه و تعیین محل استقرار مبارزان بر اساس اطلاعات ذخیره‌شده در این کلاس انجام می‌شود.

بررسی توابع کلاس

Space

تابع

`()Space`

وظیفه

ایجاد و مقداردهی اولیه یک فضای جدید از صفحه بازی.

عملکرد

تمام اطلاعات مربوط به فضای بازی را از ورودی دریافت کرده و در متغیرهای عضو کلاس ذخیره می‌کند.

محل استفاده در پروژه

هنگام ایجاد نقشه و ساخت مجموعه فضاهای صفحه بازی.

منطق پیاده‌سازی

سازنده با استفاده از لیست مقداردهی اولیه، تمامی ویژگی‌های شیء را مقداردهی می‌کند. همچنین برای انتقال بردارها از `std::move` استفاده شده است تا از کپی غیرضروری داده‌ها جلوگیری شود.

تابع

`id()`

وظیفه

برگرداندن شناسه فضای بازی.

عملکرد

شناسه یکتای فضا را در اختیار سایر بخش‌های پروژه قرار می‌دهد.

محل استفاده در پروژه

در کلاس `Board` و سایر بخش‌هایی که نیاز به شناسایی فضا دارند.

منطق پیاده‌سازی

مقدار متغیر `_id` بدون هیچ تغییری بازگردانده می‌شود.

تابع

`row()`

وظیفه

دریافت شماره سطر فضای بازی.

عملکرد

مختصات عمودی فضای موردنظر را برمی‌گرداند.

محل استفاده در پروژه

در رسم نقشه و نمایش موقعیت فضاها.

منطق پیاده‌سازی

مقدار ذخیره‌شده در `row` مستقیماً بازگردانده می‌شود.

تابع

`column()`

وظیفه

دریافت شماره ستون فضای بازی.

عملکرد

مختصات افقی فضای بازی را بازمی‌گرداند.

محل استفاده در پروژه

در رسم صفحه بازی و محاسبه محل نمایش هر فضا.

منطق پیاده‌سازی

مقدار متغیر `column` بدون تغییر بازگردانده می‌شود.

تابع

`zones()`

وظیفه

دریافت ناحیه‌های مربوط به فضای بازی.

عملکرد

لیست ناحیه‌هایی که این فضا در آن‌ها قرار دارد را برمی‌گرداند.

محل استفاده در پروژه

در بررسی قوانین حمله و تشخیص اشتراک ناحیه‌ها.

منطق پیاده‌سازی

یک ارجاع ثابت به بردار `_zones` بازگردانده می‌شود تا از ایجاد کپی جلوگیری شود.

تابع

()adjacent

وظیفه

دریافت فضاهای مجاور.

عملکرد

شناسه تمام فضاهای متصل به این فضا را برمی‌گرداند.

محل استفاده در پروژه

در محاسبه مسیر حرکت و بررسی همسایگی.

منطق پیاده‌سازی

ارجاع ثابت به بردار `_adjacent` بازگردانده می‌شود.

تابع

()hasSecretPassage

وظیفه

بررسی وجود مسیر مخفی.

عملکرد

مشخص می‌کند که آیا این فضا دارای مسیر مخفی است یا خیر.

محل استفاده در پروژه

در قوانین حرکت و نمایش نقشه.

منطق پیاده‌سازی

مقدار متغیر بولی `secretPassage` بازگردانده می‌شود.

تابع

`startSlot()`

وظیفه

دریافت شماره محل شروع.

عملکرد

شماره محل شروع تعریف‌شده برای این فضا را برمی‌گرداند.

محل استفاده در پروژه

در مرحله آغاز بازی و تعیین محل استقرار اولیه بازیکنان.

منطق پیاده‌سازی

مقدار `startSlot` مستقیماً بازگردانده می‌شود.

تابع

`zoneLabel()`

وظیفه

تولید نمایش متنی ناحیه‌های فضا.

عملکرد

تمام ناحیه‌های ذخیره‌شده را به یک رشته قابل نمایش تبدیل می‌کند.

محل استفاده در پروژه

در رابط کاربری و نمایش اطلاعات هر فضای بازی.

منطق پیاده‌سازی

ابتدا یک رشته خالی ایجاد می‌شود. سپس تمام ناحیه‌های موجود در **zones** به ترتیب پیمایش شده و در صورت وجود بیش از یک ناحیه، بین آن‌ها کاراکتر **/** قرار می‌گیرد. در نهایت رشته ساخته‌شده بازگردانده می‌شود.

GameController

هدف فایل

این فایل وظیفه مدیریت کامل جریان اجرای بازی را بر عهده دارد. کلاس **GameController** هسته اصلی منطق بازی محسوب می‌شود و مسئول راه‌اندازی بازی، مدیریت نوبت‌ها، حرکت مبارزان، اجرای حملات، اجرای کارت‌های **Scheme**، کنترل قوانین بازی، بررسی شرایط پایان بازی و هماهنگی میان سایر کلاس‌های پروژه است.

ساختارهای تعریف‌شده

ساختار

SchemeChoice

هدف و مسئولیت

این ساختار اطلاعات موردنیاز برای اجرای کارت‌های نوع **Scheme** را نگهداری می‌کند. با توجه به نوع کارت، یکی یا چند عضو این ساختار مورد استفاده قرار می‌گیرد.

ویژگی‌ها

نام	توضیح
destinationSpace	شماره فضای مقصد انتخاب‌شده
targetFighterId	شناسه مبارز هدف
namedValue	مقدار عددی انتخاب‌شده توسط بازیکن
opponentHandIndex	شماره کارت انتخاب‌شده از دست حریف

نقش در پروژه

به عنوان ورودی متد `playScheme()` استفاده می‌شود.

ساختار

PendingMovementChoice

هدف و مسئولیت

اطلاعات مربوط به حرکت اختیاری که باید بعداً انجام شود را نگهداری می‌کند.

ویژگی‌ها

نام	توضیح
playerIndex	شماره بازیکن

شناسه مبارز	fighterId
حداکثر تعداد حرکت مجاز	maxSteps
منبع ایجاد حرکت اختیاری	source

نقش در پروژه

برای مدیریت حرکتهایی که توسط برخی کارت‌ها ایجاد می‌شوند استفاده می‌شود.

کلاس

GameController

هدف و مسئولیت کلاس

این کلاس کنترل‌کننده اصلی بازی است و تقریباً تمام قوانین و روند اجرای بازی را مدیریت می‌کند. مدیریت وضعیت بازی، نوبت‌ها، کارت‌ها، مبارزه، حرکت، بررسی برنده و تعامل میان بازیکنان همگی در این کلاس انجام می‌شود.

ویژگی‌های کلاس (Attributes)

نام	توضیح
_board	صفحه بازی
_players	اطلاعات دو بازیکن

شماره بازیکن فعال	_currentPlayerIndex
تعداد اکشن باقی مانده در نوبت	_actionsRemaining
شماره نوبت	_turnNumber
وضعیت شروع بازی	_started
وضعیت پایان بازی	_gameOver
نام برنده	_winnerName
وضعیت استفاده از قابلیت ویژه دراکولا	_draculaAbilityUsed
مولد اعداد تصادفی	_random
امتیاز حرکت باقی مانده هر مبارز	_remainingMovementPoints
ثبت حرکت های انجام شده	_movedThisManeuver
مبارزانی که حرکتشان پایان یافته است	_finishedFighters
امتیاز حرکت در حال انتظار	_pendingMovementPoints
حداکثر امتیاز حرکت	_maxMovementPoints

صف حرکتهای اختیاری	<code>_pendingOptionalMovements</code>
--------------------	--

متدهای کلاس

`()GameController`

وظیفه

سازنده کلاس.

عملکرد

مقداردهی اولیه وضعیت بازی، ایجاد صفحه بازی و مقداردهی متغیرهای مدیریتی.

محل استفاده

در زمان ایجاد کنترل کننده بازی.

منطق پیاده سازی

با استفاده از `BoardFactory` صفحه بازی ساخته شده و وضعیت اولیه بازی تنظیم می شود.

`()startNewGame`

وظیفه

راه اندازی کامل یک بازی جدید.

عملکرد

- اعتبارسنجی ورودی ها
- تعیین قهرمان هر بازیکن
- ایجاد بازیکنان
- ساخت مبارزان
- ساخت دسته کارت ها
- بر زدن کارت ها
- قرار دادن مبارزان
- توزیع کارت های اولیه

- تنظیم وضعیت اولیه بازی

محل استفاده

در ابتدای شروع بازی.

منطق پیاده‌سازی

تمام وضعیت قبلی پاک شده و بازی از ابتدا مقداردهی می‌شود.

()started

بررسی می‌کند آیا بازی آغاز شده است یا خیر.

()gameOver

وضعیت پایان بازی را برمی‌گرداند.

()winnerName

نام برنده بازی را برمی‌گرداند.

()board

مرجع صفحه بازی را باز می‌گرداند.

()currentPlayer

بازیکن فعال را برمی‌گرداند.

()opponentPlayer

بازیکن مقابل را برمی‌گرداند.

()players

لیست بازیکنان بازی را باز می‌گرداند.

()currentPlayerIndex

شماره بازیکن فعال را بر می‌گرداند.

()opponentPlayerIndex

شماره بازیکن مقابل را محاسبه می‌کند.

()actionsRemaining

تعداد اکشن باقی‌مانده را باز می‌گرداند.

()turnNumber

شماره نوبت جاری را باز می‌گرداند.

()draculaAbilityAvailable

وظیفه

بررسی امکان استفاده از قابلیت ویژه دراکولا.

عملکرد

شرایط لازم مانند شروع بازی، پایان نیافتن بازی، استفاده نشدن قابلیت و وجود مبارز مجاور بررسی می‌شود.

()drawCardForCurrentPlayer

یک کارت برای بازیکن فعال می‌کشد.

()beginManeuver

وظیفه

شروع مرحله حرکت.

عملکرد

- بررسی قوانین
- کشیدن کارت
- اعمال Boost
- محاسبه امتیاز حرکت تمام مبارزان

()movableCurrentFighterIds

شناسه مبارزانی که امکان حرکت دارند را برمی‌گرداند.

()reachableDestinationsFor

فضاهای قابل دسترس برای یک مبارز را محاسبه می‌کند.

()moveCurrentFighter

وظیفه

جاب‌جایی یک مبارز.

عملکرد

اعتبار مقصد، هزینه حرکت و امتیاز باقی‌مانده بررسی شده و سپس مکان مبارز تغییر می‌کند.

()finishManeuver

مرحله حرکت را پایان داده و یک اکشن مصرف می‌کند.

()legalAttackers

تمام مبارزان مجاز برای حمله را برمی‌گرداند.

()legalTargetsFor

اهداف قانونی برای حمله را مشخص می‌کند.

()legalAttackCardsFor

کارت‌های حمله قابل استفاده را تعیین می‌کند.

()legalDefenseCardsFor

کارت‌های دفاع قابل استفاده را تعیین می‌کند.

()resolveAttack

وظیفه

اجرای کامل فرآیند مبارزه.

عملکرد

این متد مراحل زیر را انجام می‌دهد:

- بررسی اعتبار حمله
- انتخاب کارت‌ها
- اعمال اثر کارت‌ها
- محاسبه قدرت حمله و دفاع
- اعمال آسیب

- اجرای افکتهای پس از مبارزه
- انتقال کارتها به Discard
- بررسی حذف مبارزان
- بررسی پایان بازی
- کاهش تعداد اکشنها

محل استفاده

در زمان اجرای عملیات حمله.

منطق پیاده‌سازی

تمام قوانین مبارزه و افکت کارتها به صورت مرحله‌ای اجرا می‌شوند.

()legalSchemeCards

کارت‌های Scheme قابل اجرا را برمی‌گرداند.

()requiredChoiceForScheme

نوع انتخاب موردنیاز هر کارت Scheme را مشخص می‌کند.

()destinationChoicesForScheme

مقصدهای مجاز برای اجرای کارت Scheme را محاسبه می‌کند.

()targetChoicesForScheme

اهداف قابل انتخاب برای کارت‌های Scheme را تعیین می‌کند.

()namedValueChoicesForScheme

مقادیر عددی مجاز برای کارت Confirm Suspicion را باز می‌گرداند.

()opponentHandChoicesForScheme

شماره کارت‌های قابل انتخاب از دست حریف را برمی‌گرداند.

()playScheme

وظیفه

اجرای کارت‌های Scheme.

عملکرد

ابتدا اعتبار ورودی بررسی شده و سپس بر اساس نوع Effect، عملیات متناظر مانند جابه‌جایی، درمان، آسیب، حذف کارت یا جابه‌جایی مبارزان انجام می‌شود.

()legalBoostCardIndexes

تمام کارت‌های قابل استفاده برای Boost را باز می‌گرداند.

()discardCurrentPlayerCard

یک کارت را از دست بازیکن حذف کرده و به دسته کارت‌های مصرف‌شده منتقل می‌کند.

()useDraculaStartAbility

قابلیت ویژه ابتدای نوبت دراکولا را اجرا می‌کند.

()endTurnIfNeeded

در صورت پایان اکشن‌ها، نوبت را خاتمه می‌دهد.

()currentPlayerMustDiscardToLimit

بررسی می‌کند آیا بازیکن باید برای رعایت محدودیت دست، کارت حذف کند یا خیر.

()hasPendingOptionalMovement

وجود حرکت اختیاری در انتظار را بررسی می‌کند.

()pendingOptionalMovement

اطلاعات حرکت اختیاری جاری را برمی‌گرداند.

()pendingOptionalMovementDestinations

مقصدهای قابل انتخاب برای حرکت اختیاری را محاسبه می‌کند.

()resolvePendingOptionalMovement

حرکت اختیاری ذخیره‌شده را اجرا یا لغو می‌کند.

()occupantTokens

نماد نمایشی تمام مبارزان روی صفحه را تولید می‌کند.

()findFighterById

جستجوی مبارز بر اساس شناسه.

()ownerOfFighter

مالک یک مبارز را مشخص می‌کند.

()playerByIndex

بازیکن متناظر با شماره مشخص را بازی می‌گرداند.

()ownerOfFighterMutable

نسخه قابل تغییر مالک مبارز را بازی می‌گرداند.

()opponentOf

بازیکن مقابل یک بازیکن را برمی‌گرداند.

()isCardPlayableBy

بررسی می‌کند آیا کارت توسط مبارز موردنظر قابل استفاده است یا خیر.

()canAttackTarget

بررسی امکان حمله به هدف بر اساس فاصله و نوع حمله.

()isSpaceOccupied

اشغال بودن یک فضای صفحه را بررسی می‌کند.

()advanceTurn

نوبت را به بازیکن بعدی منتقل کرده و وضعیت نوبت جدید را مقداردهی می‌کند.

()drawCard

یک کارت از Deck به Hand منتقل می‌کند و در صورت خالی بودن Deck قانون Fatigue را اجرا می‌کند.

()fatigue

در صورت اتمام Deck به تمام مبارزان زنده بازیکن آسیب وارد می‌کند.

()placeSidekicks

یاران قهرمان را در خانه‌های آزاد اطراف قهرمان قرار می‌دهد.

()checkDefeatedFighters

مبارزان شکست‌خورده را از صفحه حذف می‌کند.

()checkWinner

در صورت شکست قهرمان، پایان بازی و برنده را تعیین می‌کند.

()shuffleDeck

دسته کارت‌های بازیکن را به صورت تصادفی بر می‌زند.

()countLivingSistersInZoneWith

تعداد خواهرهای زنده موجود در ناحیه مشخص را محاسبه می‌کند.

()moveFighterIgnoringDistance

جابه‌جایی مستقیم مبارز بدون در نظر گرفتن محدودیت فاصله.

()reachableForPlayerFighter

خانه‌های قابل دسترس برای یک مبارز را محاسبه می‌کند.

()queueOptionalMovement

یک حرکت اختیاری را در صف انتظار ثبت می‌کند.

()freeSpacesSharingHeroZone

خانه‌های آزاد موجود در ناحیه قهرمان را برمی‌گرداند.

()valuesOnCard

مقادیر حمله و دفاع موجود روی کارت را استخراج می‌کند.

()cardEffectsProtectedBySherlock

بررسی می‌کند آیا افکت کارت تحت محافظت قوانین شرلوک قرار دارد یا خیر.

()resolveCombatEffectAfterDamage

تمام افکت‌هایی که پس از اعمال آسیب باید اجرا شوند را مدیریت می‌کند.

()pendingMovementPoints

مجموع امتیاز حرکت باقی‌مانده را محاسبه می‌کند.

()maxMovementPoints

حداکثر امتیاز حرکت مبارزان بازیکن را محاسبه می‌کند.

()finishCurrentFighter

حرکت یک مبارز را خاتمه‌یافته علامت‌گذاری می‌کند.

()getMovementCost

هزینه رسیدن یک مبارز به مقصد مشخص را محاسبه می‌کند.

()remainingMovementForFighter

امتیاز حرکت باقی‌مانده یک مبارز را باز می‌گرداند.

()computeReachableWithCost

خانه‌های قابل دسترس و هزینه رسیدن به هر خانه را با در نظر گرفتن موانع و اشغال بودن خانه‌ها محاسبه می‌کند.

()getMatchingCardIndicesForConfirmSuspicion

کارت‌های دست حریف که دارای مقدار انتخاب‌شده هستند را پیدا می‌کند.

()applyConfirmSuspicion

اثر کارت Confirm Suspicion را اجرا کرده، کارت انتخاب‌شده را حذف و آسیب متناظر را به قهرمان حریف وارد می‌کند.

ارتباط با سایر کلاس‌ها

کلاس **GameController** با کلاس‌های زیر در ارتباط مستقیم است:

- **Board** برای مدیریت صفحه بازی و محاسبات مسیرها
- **Player** برای مدیریت اطلاعات بازیکنان
- **Fighter** برای کنترل وضعیت مبارزان
- **Card** برای اجرای کارت‌ها و افکت‌های آن‌ها
- **BoardFactory** برای ایجاد صفحه بازی
- **DeckFactory** برای ایجاد دسته کارت‌ها
- **FighterFactory** برای ایجاد مبارزان
- **GameExceptions** برای مدیریت خطاهای مربوط به قوانین بازی

نقش کلاس در عملکرد کلی پروژه

کلاس **GameController** هسته اصلی منطق پروژه محسوب می‌شود. تمامی عملیات مهم شامل آغاز بازی، مدیریت نوبت‌ها، حرکت مبارزان، اجرای حملات، اعمال افکت کارت‌ها، مدیریت قوانین، کنترل وضعیت بازیکنان و تعیین پایان بازی توسط این کلاس انجام می‌شود و سایر کلاس‌های پروژه اطلاعات و امکانات موردنیاز خود را در اختیار این کنترل‌کننده قرار می‌دهند.

TuiApp

هدف فایل

این فایل رابط کاربری متنی (Text User Interface) پروژه را پیاده‌سازی می‌کند. کلاس **TuiApp** مسئول نمایش صفحات مختلف بازی، دریافت ورودی کاربر، مدیریت منوها و برقراری ارتباط میان کاربر و منطق اصلی بازی است.

این کلاس مستقیماً قوانین بازی را پیاده‌سازی نمی‌کند، بلکه از طریق کلاس **GameController** عملیات مربوط به بازی را فراخوانی کرده و نتیجه را در محیط متنی نمایش می‌دهد.

کلاس TuiApp

هدف و مسئولیت کلاس

کلاس **TuiApp** رابط کاربری متنی پروژه را مدیریت می‌کند. مهمترین مسئولیت‌های آن عبارت‌اند از:

- نمایش صفحات مختلف برنامه
- مدیریت منوها
- دریافت ورودی‌های صفحه‌کلید
- هدایت مراحل راه‌اندازی بازی
- هدایت جریان اجرای نوبت‌ها
- نمایش وضعیت بازی

- مدیریت خطاهای رابط کاربری
- برقراری ارتباط میان کاربر و `GameController`

ویژگی‌های کلاس `Attributes()`

ح	
مرجع به صفحه تعاملی FTXUI برای نمایش رابط کاربری	<code>_screen</code>
شیء کنترل‌کننده اصلی بازی	<code>_controller</code>
وضعیت فعلی رابط کاربری	<code>_state</code>
شماره گزینه انتخاب شده در منو	<code>_selected</code>
رشته مربوط به ورود سن بازیکنان	<code>_ageInput</code>
مشخص می‌کند ورود سن مربوط به کدام بازیکن است	<code>_ageStep</code>
سن بازیکن اول	<code>_playerOneAge</code>
سن بازیکن دوم	<code>_playerTwoAge</code>
قهرمان انتخاب‌شده	<code>_selectedHero</code>

محل شروع انتخاب‌شده	<code>_selectedStartSlot</code>
پیام خطا برای نمایش در رابط کاربری	<code>_errorMessage</code>
تعیین حالت نمایش ASCII	<code>_asciiOnlyMode</code>
فهرست مبارزان قابل انتخاب در مراحل مختلف	<code>_pendingFighterIds</code>
فهرست کارت‌های قابل انتخاب	<code>_pendingCardIndexes</code>
فهرست مکان‌های قابل انتخاب	<code>_pendingSpaces</code>
فهرست مقادیر عددی قابل انتخاب	<code>_pendingValues</code>
شناسه مبارز حمله‌کننده	<code>_selectedAttackerId</code>
شناسه هدف حمله	<code>_selectedTargetId</code>
مقدار انتخاب‌شده برای برخی کارت‌های خاص	<code>_selectedNamedValue</code>
کارت حمله انتخاب‌شده	<code>_selectedAttackCardIndex</code>
کارت دفاع انتخاب‌شده	<code>_selectedDefenseCardIndex</code>
کارت Scheme انتخاب‌شده	<code>_selectedSchemeCardIndex</code>

کارت‌های تقویت Beast Form	<code>_selectedBeastFormBoostIndexes</code>
اطلاعات انتخاب‌های موردنیاز کارت‌های <code>scheme</code>	<code>_schemeChoice</code>

ارتباط کلاس با سایر کلاس‌ها

کلاس `TuiApp` با کلاس‌های زیر ارتباط مستقیم دارد:

- `GameController`
- `Player`
- `Card`
- `Fighter`
- `Board`
- `Space`
- `GameException`
- اجزای کتابخانه `FTXUI`

تمام عملیات منطقی بازی از طریق `GameController` انجام می‌شود و `TuiApp` تنها نقش واسط کاربری را ایفا می‌کند.

متدهای کلاس

`( )TuiApp`

وظیفه

سازنده کلاس.

عملکرد

مقادیر اولیه رابط کاربری، وضعیت اولیه بازی و متغیرهای انتخاب را مقداردهی می‌کند.

محل استفاده

در زمان ایجاد برنامه.

()Render

وظیفه

رندر صفحه فعلی.

عملکرد

بر اساس مقدار **ScreenState** صفحه مناسب را انتخاب کرده و عنصر گرافیکی مربوطه را باز می‌گرداند.

()OnEvent

وظیفه

دریافت رویدادهای صفحه‌کلید.

عملکرد

رویداد را به **handleEvent()** ارسال کرده و در صورت وقوع استثناء، پیام خطا را نمایش می‌دهد.

()renderMainMenu

وظیفه

نمایش منوی اصلی.

عملکرد

صفحه آغازین برنامه شامل گزینه‌های شروع بازی، راهنما و خروج را ایجاد می‌کند.

()renderHelp

وظیفه

نمایش راهنمای بازی.

عملکرد

قوانین اصلی بازی را به صورت متنی نمایش می‌دهد.

()renderSetupAge

وظیفه

دریافت سن بازیکنان.

عملکرد

ورودی عددی سن هر بازیکن را از کاربر دریافت می‌کند.

()renderFighterSelect

وظیفه

انتخاب شخصیت.

عملکرد

امکان انتخاب یکی از دو شخصیت بازی را فراهم می‌کند.

()renderStartSelect

وظیفه

انتخاب محل شروع.

عملکرد

محل اولیه قرارگیری شخصیت‌ها را از کاربر دریافت می‌کند.

()renderGame

وظیفه

نمایش صفحه اصلی بازی.

عملکرد

تمام اجزای بازی شامل:

- اطلاعات نوبت
- وضعیت بازیکنان
- نقشه
- کارت‌ها
- منوی عملیات

را در یک صفحه نمایش می‌دهد.

renderGameOver()

وظیفه

نمایش پایان بازی.

عملکرد

برنده بازی را نمایش داده و امکان بازگشت یا خروج را فراهم می‌کند.

renderPlayerPanel()

وظیفه

نمایش اطلاعات یک بازیکن.

عملکرد

نمایش:

- نام
- شخصیت
- وضعیت فعال بودن
- مبارزان
- تعداد کارت‌ها
- Deck
- Discard

()renderHandPanel

وظیفه

نمایش کارت‌های دست بازیکن.

عملکرد

حداکثر هشت کارت را همراه اطلاعات آن‌ها نمایش می‌دهد.

()renderActionPanel

وظیفه

نمایش منوی عملیات.

عملکرد

عنوان منو را بر اساس وضعیت فعلی بازی تعیین کرده و گزینه‌های موجود را نمایش می‌دهد.

()renderMapPanel

وظیفه

نمایش نقشه.

عملکرد

اطلاعات برد بازی را از **GameController** دریافت و به صورت متنی نمایش می‌دهد.

()renderMenuLines

وظیفه

نمایش گزینه‌های منو.

عملکرد

تمام گزینه‌ها را همراه گزینه انتخاب‌شده رسم می‌کند.

handleEvent()

وظیفه

مدیریت رویدادهای صفحه‌کلید.

عملکرد

ورودی‌هایی مانند:

- جهت‌ها
- Enter
- Escape
- اعداد
- Backspace
- Q

را مدیریت کرده و عملیات مناسب را اجرا می‌کند.

handleEnter()

وظیفه

پردازش کلید Enter.

عملکرد

بخش اصلی کنترل جریان برنامه است و تقریباً تمام وضعیت‌های رابط کاربری را مدیریت می‌کند؛ از جمله:

- شروع بازی
- انتخاب شخصیت
- انتخاب محل شروع
- Maneuver
- Attack
- Defense
- Scheme
- پایان بازی
- دور ریختن کارت
- استفاده از قابلیت Dracula

()handleDigit

وظیفه

ورود عدد.

عملکرد

ارقام واردشده برای سن بازیکنان را ذخیره می‌کند.

()handleBackspace

وظیفه

حذف آخرین رقم واردشده.

()moveSelection

وظیفه

جابجایی انتخاب منو.

عملکرد

گزینه انتخاب‌شده را بالا یا پایین می‌برد.

()currentMenuEntries

وظیفه

تولید گزینه‌های منوی فعلی.

عملکرد

بر اساس وضعیت فعلی بازی، گزینه‌های مجاز را تولید می‌کند.

()resetSelection

وظیفه

بازنشانی انتخاب منو.

()showError

وظیفه

نمایش پیام خطا.

()openGameScreen

وظیفه

بازگشت به صفحه اصلی بازی.

عملکرد

پس از پایان عملیات مختلف، وضعیت مناسب بعدی را تعیین می‌کند؛ مانند:

- ادامه بازی
- پایان بازی
- اجبار به دور ریختن کارت
- حرکت اختیاری

()startGameFromSetup

وظیفه

شروع بازی.

عملکرد

پارامترهای انتخاب‌شده توسط کاربر را به **GameController** ارسال کرده و بازی را آغاز می‌کند.

()beginAttackFlow

وظیفه

شروع فرآیند حمله.

()beginSchemeFlow

وظیفه

شروع فرآیند اجرای کارت‌های Scheme.

()beginManeuverFlow

وظیفه

شروع فرآیند Maneuver.

()chooseSchemeCard

وظیفه

انتخاب کارت Scheme.

عملکرد

بررسی می‌کند کارت انتخاب‌شده به چه نوع انتخاب اضافی نیاز دارد و وضعیت مناسب را ایجاد می‌کند.

()completeSchemeChoice

وظیفه

تکمیل انتخاب‌های موردنیاز کارت‌های Scheme.

عملکرد

با توجه به نوع انتخاب موردنیاز، اطلاعات لازم را دریافت کرده و اجرای کارت را به **GameController** واگذار می‌کند.

()fighterMenuLabel

وظیفه

تولید متن نمایشی یک مبارز.

()cardMenuLabel

وظیفه

تولید متن نمایشی یک کارت.

()spaceMenuLabel

وظیفه

تولید متن نمایشی یک فضای نقشه.

()menuLine

وظیفه

رسم یک گزینه منو.

()cardElement

وظیفه

رسم اطلاعات یک کارت در رابط کاربری.

()fighterLine

وظیفه

رسم اطلاعات یک مبارز.

عملکرد

اطلاعاتی مانند:

- نام
- سلامتی
- نوار HP
- موقعیت
- وضعیت شکست

را نمایش می‌دهد.

heroColor()

وظیفه

تعیین رنگ شخصیت.

عملکرد

برای Dracula رنگ قرمز و برای Sherlock رنگ آبی باز می‌گرداند.

توابع کمکی

repeat()

تولید رشته‌ای شامل تکرار یک کاراکتر به تعداد مشخص.

fixedWidth()

یک رشته را تا طول مشخص کوتاه یا با فاصله پر می‌کند تا نمایش منظم‌تری در رابط کاربری داشته باشد.

()hpBar

نوار گرافیکی سلامت مبارز را بر اساس مقدار فعلی و حداکثر سلامتی تولید می‌کند.

وابستگی‌های فایل

کتابخانه‌های استاندارد

- algorithm •
- cctype •
- io manip •
- optional •
- sstream •
- string •
- vector •

فایل‌های داخلی پروژه

- GameController.hpp •
- GameExceptions.hpp •

کتابخانه خارجی

- FTXUI •

برای ایجاد رابط کاربری متنی، نمایش عناصر و مدیریت رویدادهای صفحه‌کلید استفاده شده است.

نقش کلاس در عملکرد کلی پروژه

کلاس **TuiApp** لایه رابط کاربری پروژه را تشکیل می‌دهد و تمام تعاملات کاربر با بازی از طریق این کلاس انجام می‌شود. این کلاس مسئول نمایش وضعیت بازی، دریافت ورودی‌ها، هدایت مراحل مختلف مانند انتخاب شخصیت، انجام عملیات بازی (Maneuver، Attack، Scheme)، مدیریت منوها و نمایش پیام‌های خطا است. منطق اصلی بازی را مستقیماً پیاده‌سازی نمی‌کند، بلکه تمامی عملیات را به **GameController** واگذار کرده و نتایج را در محیط متنی نمایش می‌دهد؛ بنابراین نقش آن ایجاد ارتباط میان کاربر و هسته منطقی بازی است.

GameExceptions

هدف فایل

این فایل مجموعه‌ای از کلاس‌های استثناء (Exception) اختصاصی پروژه را تعریف می‌کند. هدف آن ایجاد سازوکاری استاندارد برای مدیریت خطاهای مرتبط با بازی و تفکیک انواع مختلف خطاها است تا بخش‌های مختلف برنامه بتوانند آن‌ها را شناسایی و مدیریت کنند.

کلاس

GameException

هدف و مسئولیت کلاس

کلاس `GameException` پایه برای تمامی استثناءهای اختصاصی پروژه است. این کلاس از `std::runtime_error` ارث‌بری می‌کند و امکان ایجاد پیام خطای سفارشی را فراهم می‌سازد.

ویژگی‌ها (Attributes)

این کلاس ویژگی جدیدی تعریف نمی‌کند و تنها از ویژگی‌ها و سازوکار کلاس `std::runtime_error` استفاده می‌کند.

متدهای کلاس

`(GameException(const std::string& message`

وظیفه: ایجاد یک استثناء همراه با پیام خطا.

عملکرد: پیام دریافت‌شده را به سازنده کلاس `std::runtime_error` ارسال می‌کند تا در زمان بروز خطا قابل بازیابی و نمایش باشد.

محل استفاده در پروژه: در هر بخشی از پروژه که نیاز به ایجاد یک خطای عمومی مرتبط با بازی وجود داشته باشد.

منطق پیاده‌سازی: سازنده تنها پیام خطا را به کلاس پایه منتقل می‌کند و منطق اضافه‌ای ندارد.

ارتباط با سایر کلاس‌ها

- از کلاس `std::runtime_error` ارث‌بری می‌کند.
- کلاس‌های `InvalidSetup` و `RuleViolation` از این کلاس مشتق شده‌اند.
- در بخش‌هایی مانند رابط کاربری (TuiApp) برای مدیریت خطاها مورد استفاده قرار می‌گیرد.

نقش کلاس در عملکرد کلی پروژه

این کلاس پایه مشترک تمام خطاهای اختصاصی پروژه را فراهم می‌کند و باعث می‌شود مدیریت استثناءها به صورت یکپارچه انجام شود.

کلاس

RuleViolation

هدف و مسئولیت کلاس

این کلاس برای نمایش خطاهایی به کار می‌رود که در نتیجه نقض قوانین بازی ایجاد می‌شوند.

ویژگی‌ها (Attributes)

ویژگی جدیدی تعریف نشده است.

متدهای کلاس

`(RuleViolation(const std::string& message`

وظیفه: ایجاد استثناء مربوط به نقض قوانین بازی.

عملکرد: پیام خطا را به سازنده کلاس `GameException` ارسال می‌کند.

محل استفاده در پروژه: در بخش‌هایی که قوانین بازی بررسی می‌شوند و عملیاتی برخلاف قوانین انجام گیرد.

منطق پیاده‌سازی: تنها پیام را به کلاس پایه منتقل می‌کند.

ارتباط با سایر کلاس‌ها

- از کلاس `GameException` ارث‌بری می‌کند.
- توسط بخش‌های مربوط به منطق بازی برای اعلام نقض قوانین قابل استفاده است.

نقش کلاس در عملکرد کلی پروژه

باعث تفکیک خطاهای مربوط به قوانین بازی از سایر انواع خطا شده و مدیریت دقیق‌تر استثناءها را امکان‌پذیر می‌کند.

کلاس

InvalidSetup

هدف و مسئولیت کلاس

این کلاس برای اعلام خطاهای مربوط به راه اندازی یا پیکربندی اولیه بازی استفاده می‌شود.

ویژگی‌ها (Attributes)

ویژگی جدیدی ندارد.

متدهای کلاس

(InvalidSetup(const std::string& message

وظیفه: ایجاد استثناء مربوط به تنظیمات یا راه اندازی نامعتبر.

عملکرد: پیام خطا را به کلاس پایه ارسال می‌کند.

محل استفاده در پروژه: در بخش‌هایی که اعتبار تنظیمات اولیه بازی بررسی می‌شود.

منطق پیاده‌سازی: تنها پیام خطا را به سازنده `GameException` منتقل می‌کند.

ارتباط با سایر کلاس‌ها

- از کلاس `GameException` مشتق شده است.
- برای گزارش خطاهای مربوط به راه اندازی بازی مورد استفاده قرار می‌گیرد.

نقش کلاس در عملکرد کلی پروژه

این کلاس امکان جداسازی خطاهای مربوط به مرحله آماده‌سازی بازی از سایر خطاها را فراهم می‌کند و خوانایی و نگهداری کد را بهبود می‌بخشد.

کتابخانه‌های استاندارد

- `std::runtime_error` : برای استفاده از کلاس `std::exception`

- **string** : برای نگهداری پیام‌های خطا

فایل‌های داخلی پروژه

این فایل وابستگی به سایر فایل‌های داخلی پروژه ندارد.

چالش‌های پروژه

در طول اجرای این پروژه با چالش‌های متعددی روبه‌رو شدیم. پیاده‌سازی هر بخش از پروژه، از طراحی نقشه و شخصیت‌ها گرفته تا مدیریت کارت‌ها و قوانین بازی، چالش‌های خاص خود را داشت و نیازمند بررسی و آزمایش مداوم بود. مهمترین چالش‌های این پروژه عبارت‌اند از:

- **رعایت تجربه کاربری مناسب:** نمایش صحیح اطلاعات بازی، وضعیت مبارزان، کارت‌های بازیکنان و روند اجرای نبردها به شکلی ساده، روان و قابل فهم، یکی از مهمترین چالش‌ها بود تا بازیکنان بتوانند بدون ابهام با بازی تعامل داشته باشند.
- **حفظ هماهنگی بین اجزای بازی:** تمامی بخش‌های بازی، از جمله نقشه، کارت‌ها، شخصیت‌ها و قوانین، به یکدیگر وابسته هستند. ایجاد هماهنگی میان این اجزا و جلوگیری از بروز خطا در جریان بازی، از مهمترین چالش‌های توسعه پروژه محسوب می‌شد.
- **مدیریت تعامل کارت‌ها:** هر کارت دارای اثر و زمان اجرای متفاوتی است و برخی از آن‌ها قبل از نبرد، هنگام نبرد یا پس از نبرد فعال می‌شوند. پیاده‌سازی صحیح ترتیب اجرای این اثرها و جلوگیری از تداخل میان آن‌ها، یکی از پیچیده‌ترین بخش‌های پروژه بود.
- **مدیریت وضعیت مبارزان:** کنترل موقعیت مبارزان روی نقشه، بررسی مجاورت، مدیریت ناحیه‌ها (Zone)، گذرگاه‌های مخفی، میزان سلامتی، حذف مبارزان شکست‌خورده و اعمال صحیح قوانین حرکت، نیازمند طراحی دقیق و مدیریت مناسب وضعیت بازی بود.

علاوه بر چالش‌های مربوط به طراحی و پیاده‌سازی، در طول توسعه پروژه با خطاها و مشکلات متعددی نیز روبه‌رو شدیم. از جمله این مشکلات می‌توان به خطاهای منطقی در اجرای قوانین بازی، اشکالات مربوط به ترتیب اجرای اثر کارت‌ها، خطاهای مرتبط با جابجایی مبارزان روی نقشه، ناهماهنگی بین وضعیت بازی و رابط کاربری، و همچنین اشکالات ناشی از ارتباط بین کلاس‌ها و اجزای مختلف پروژه اشاره کرد. رفع این مشکلات با استفاده از آزمون‌های مکرر، بررسی سناریوهای مختلف بازی و اصلاح تدریجی ساختار برنامه انجام شد.

منابع

- ویدئوهای آموزشی دکتر صنعتی
- YouTube
- Google
- ChatGPT
- DeepSeek
- faradars.org